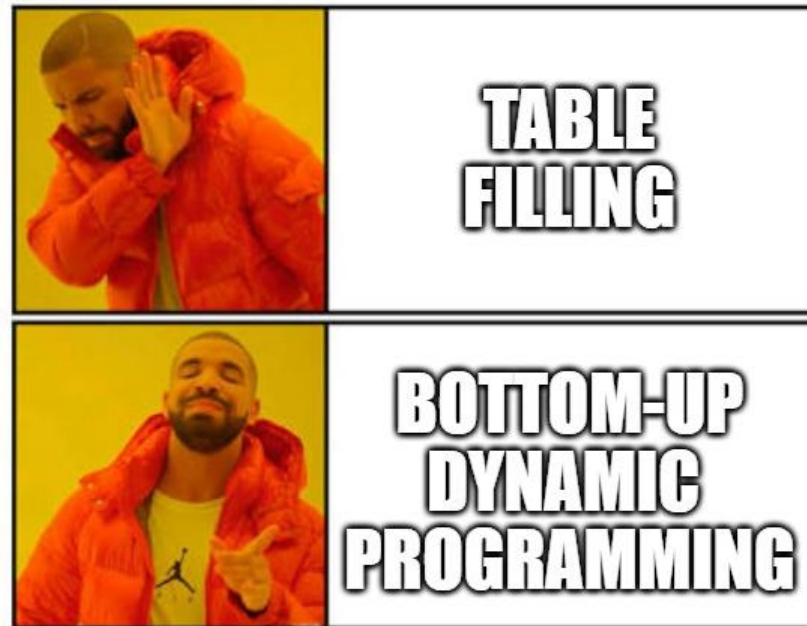


D3: Dynamic Programming



Sec 17: Fr 11:30am-12:30pm FXB 1024
IA: Pearl Lin

Agenda

- ▶ Admin
- ▶ Recurrence relations
- ▶ Dynamic programming
 - ▶ Divide & conquer vs DP
 - ▶ DP Recipe
- ▶ Worksheet

Admin

- ▶ HW 3 released, due next Wednesday 9/23 at 8PM
- ▶ We will try to get your HW scores back ~1 week after it's due

Recurrence Relations

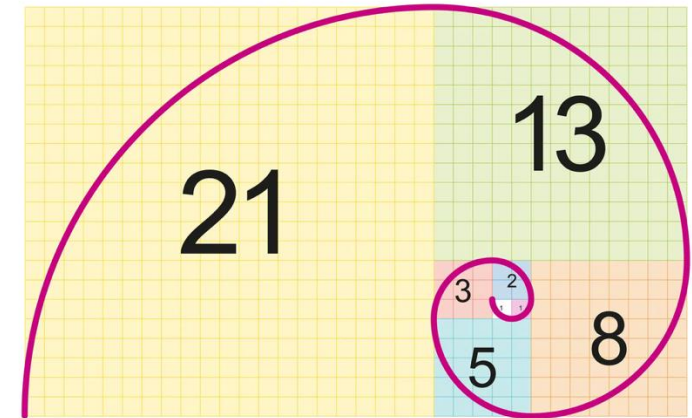
Recurrence Relations

- ▶ A function defined in terms of its **previous values**
 - ▶ Requires initial values for T to fully define the recurrence (**base case(s)**)
 - ▶ Model algorithms that depend on **subproblems** (ex. divide & conquer, dynamic programming)

$$\left\{ \begin{array}{l} T(n) = f(T(n_0), T(n_1), \dots, T(n-k)) \\ T(n_0) = T_0 \\ T(n_1) = T_1 \\ \dots \\ T(n_k) = T_k \end{array} \right. \quad \text{where } k \text{ is a constant}$$

- ▶ Example: Fibonacci

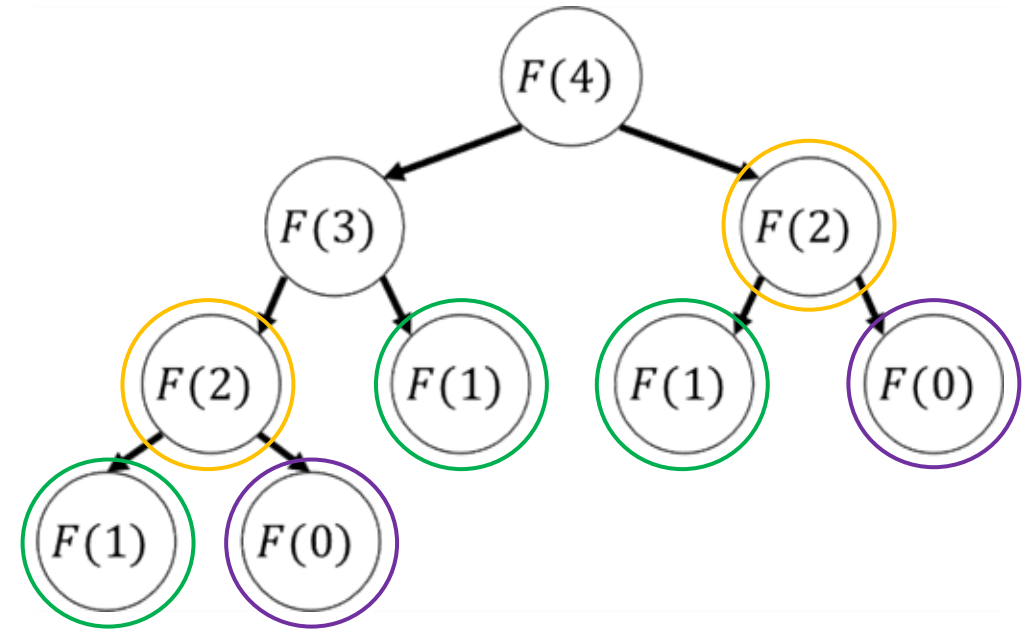
$$\left\{ \begin{array}{l} F(n) = F(n-1) + F(n-2) \\ F(0) = 0 \\ F(1) = 1 \end{array} \right.$$



Dynamic Programming

Characteristics of DP Problems

- ▶ **Principle of optimality**: optimal solution to a larger instance is made up of optimal solution(s) to one or more smaller sub-instances
- ▶ **Overlapping subproblems**: same sub-inputs occur many times while recursing down to the base case
 - ▶ Use memoization to remember what we got for these sub-inputs. Then we don't need to compute them over and over again!
 - ▶ Ex. Fibonacci



Fibonacci with re-occurring subproblems circled

Divide and Conquer v.s. DP

Divide and Conquer	Dynamic Programming
Divide original problem to smaller version(s) of the same problem	
Non-overlapping subproblems	Overlapping subproblems
Subproblems usually scale down by a constant: $T(n) \rightarrow T(\dots)$	Subproblems don't usually scale down: $T(n) \rightarrow T(n-1) \rightarrow T(n-2) \rightarrow \dots$
Optimal substructure: The solution is correct for this (scaled-down) portion	Optimal substructure: The solution is correct up to this point
Always top-town	Top-down, memoization, bottom-up
Often less time efficient	Often more time efficient- especially with bottom-up

Why is MergeSort a divide and conquer algorithm rather than a DP algorithm?

Divide and Conquer v.s. DP

Divide and Conquer	Dynamic Programming
Divide original problem to smaller version(s) of the same problem	
Non-overlapping subproblems	Overlapping subproblems
Subproblems usually scale down by a constant: $T(n) \rightarrow T(n-1) \rightarrow \dots$	Subproblems don't usually scale down: $T(n) \rightarrow T(n-1) \rightarrow T(n-2) \rightarrow \dots$
Optimal substructure: The solution is correct for this (scaled-down) portion	Optimal substructure: The solution is correct up to this point
Always top-down	Top-down, memoization, bottom-up
Often less time efficient	Often more time efficient- especially with bottom-up

Why is MergeSort a divide and conquer algorithm rather than a DP algorithm?

No overlapping subproblems. Once a section of the array is sorted you never need to sort it again

DP Recipe



1. Define and focus on the “value version” of the problem

- ▶ “Let **[name of recurrence]** be **[definition of recurrence]**”
- ▶ For optimization-style problems, focus on the **value being optimized**, not on the optimal solution itself
- ▶ Ex. “Let **$END_LIS(i)$** be the **length of the longest increasing subsequence ending with $S[i]$** ”

2. Devise a recurrence for the value in question

- ▶ Oftentimes the **hardest** part
- ▶ How is the solution made up of subproblem solutions?
- ▶ What are the base cases?

3. Implement the recurrence in (pseudo)code

- ▶ Create and fill up a memoization table in a bottom-up fashion

DP Recipe (continued)



4. If applicable, extend the (pseudo)code to solve the original problem using “backtracking”

- ▶ Now that you have the **optimal value**, it’s time to get back to the **optimal solution** associated with that value
- ▶ Idea: leave “breadcrumbs” that you can follow to get back to the optimal solution

5. Perform runtime analysis

- ▶ What is the # of table entries being filled?
- ▶ How much time does it take to fill each entry?
- ▶ Generally:

$$\text{Runtime} = (\# \text{ of table entries}) \cdot (\text{time to fill each entry})$$

Tips for Solving DP Problems

- ▶ DP is very tricky. Although there is no set formula for solving DP problems, here are some general tips:
 1. Focus on just the ***i*th entity** when reasoning about the recurrence
 - ▶ How does the addition of the *i*th entity affect (or not affect) the optimal solution?
 2. When you can't find a relationship between subproblems, try to **restrict your view** of the original problem
 - ▶ Example: LIS. instead of finding $LIS(i)$, we find $END_LIS(i)$
 - ▶ Redefine the recurrence if necessary
 3. Pattern recognition
 - ▶ There are only so many different problems you can see before you start noticing **similarities**
 - ▶ Identify if the new problem resembles something you have solved before and apply the same techniques!
 - ▶ **Make sure you understand all example problems from lecture!!!**
 - ▶ Comes easier with practice/experience

Worksheet

Q1

1. Climb Stairs

We define the function $\text{CLIMB-STAIRS}(n)$ as follows: There are n steps to climb. Return the number of ways there are for a person standing at the bottom to reach the top if they can climb either 1, 2 or 3 steps at a time.

Devise a recurrence for computing $\text{CLIMB-STAIRS}(n)$.

Q1 (annotations)

1. Climb Stairs

We define the function $\text{CLIMB-STAIRS}(n)$ as follows: There are n steps to climb. Return the number of ways there are for a person standing at the bottom to reach the top if they can climb either 1, 2 or 3 steps at a time.

Devise a recurrence for computing $\text{CLIMB-STAIRS}(n)$.

Q1 (solution)

1. Climb Stairs

We define the function $\text{CLIMB-STAIRS}(n)$ as follows: There are n steps to climb. Return the number of ways there are for a person standing at the bottom to reach the top if they can climb either 1, 2 or 3 steps at a time.

Devise a recurrence for computing $\text{CLIMB-STAIRS}(n)$.

Recursive case: Let's consider just the last step in the climb.
There are 3 possible cases:

Case 1: size of last step = 1

Path = path from bottom to stair $(n-1)$ + last step of size 1

$$\Rightarrow T(n) = T(n-1)$$

Case 2: size of last step = 2

Path = path from bottom to stair $(n-2)$ + last step of size 2

$$\Rightarrow T(n) = T(n-2)$$

Case 3: size of last step = 3

Path = path from bottom to stair $(n-3)$ + last step of size 3

$$\Rightarrow T(n) = T(n-3)$$

Sum it all together: $T(n) = T(n-1) + T(n-2) + T(n-3)$

Base cases:

Ways to climb 0 stairs = 1

$$\Rightarrow T(0) = 1$$

Ways to climb 1 stair = 1

$$\Rightarrow T(1) = 1$$

Ways to climb 2 stairs = 2

$$\Rightarrow T(2) = 2$$

Q2

Hint: let's say that the only meal available has 5 chicken nuggets. If you are able to purchase exactly t nuggets, what can you say about whether you are able to purchase $t-5$ nuggets?

2. McNuggets

We have a collection M of chicken McNuggets meals; these meals are displayed to you in a menu, represented as an array $M[1, \dots, n]$, with the number of McNuggets per meal. Your goal is to determine, for a given positive integer t , whether it is possible to purchase *exactly* t McNuggets.¹ For example, for $M = [4, 6, 10, 20]$ and $t = 22$, it is possible with $M[2] + M[2] + M[4] = 22$; however, for the same M and $t = 13$, it is not possible.²

Give a recurrence relation (including base cases), that is suitable for a dynamic programming solution to solve this problem in $O(t \cdot n)$ time. Your solution should include an explanation of why the recurrence is correct.

Q2 (annotations)

2. McNuggets

We have a collection M of chicken McNuggets meals; these meals are displayed to you in a menu, represented as an array $M[1, \dots, n]$, with the number of McNuggets per meal. Your goal is to determine, for a given positive integer t , whether it is possible to purchase *exactly* t McNuggets.¹ For example, for $M = [4, 6, 10, 20]$ and $t = 22$, it is possible with $M[2] + M[2] + M[4] = 22$; however, for the same M and $t = 13$, it is not possible.²

Give a recurrence relation (including base cases), that is suitable for a dynamic programming solution to solve this problem in $O(t \cdot n)$ time. Your solution should include an explanation of why the recurrence is correct.

Q2 (solution)

2. McNuggets

We have a collection M of chicken McNuggets meals; these meals are displayed to you in a menu, represented as an array $M[1, \dots, n]$, with the number of McNuggets per meal. Your goal is to determine, for a given positive integer t , whether it is possible to purchase *exactly* t McNuggets.¹ For example, for $M = [4, 6, 10, 20]$ and $t = 22$, it is possible with $M[2] + M[2] + M[4] = 22$; however, for the same M and $t = 13$, it is not possible.²

Give a recurrence relation (including base cases), that is suitable for a dynamic programming solution to solve this problem in $O(t \cdot n)$ time. Your solution should include an explanation of why the recurrence is correct.

Recursive case:

For any meal $M[k]$, if I'm able to buy exactly $t - M[k]$ nuggets using meals, then I can definitely buy exactly t nuggets using meals. All I need to do is to buy $t - M[k]$ nuggets and add on the last meal $M[k]$.

If no meals on the menu can meet this condition at all, we're out of luck. It means that we cannot attain the value using an integer number of each meal.

$$\text{McNuggets}(i) = \bigvee_{j=1}^n \text{McNuggets}(i - M[j])$$

Base cases:

$i = 0$: True since we can always buy exactly 0 nuggets by not buying any meals

$i < 0$: False since we cannot attain a negative number of nuggets

Q4

Hint: what lecture example is similar to this?

4. Longest arithmetic subsequence.

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

LONGEST-ARITHMETIC-SUBSEQUENCE(A, d): Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest subsequence S of A such that $S[i + 1] - S[i] = d$ for each i .

Q4 (annotations)

4. Longest arithmetic subsequence.

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

LONGEST-ARITHMETIC-SUBSEQUENCE(A, d): Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest subsequence S of A such that $S[i + 1] - S[i] = d$ for each i .

Q4 (solution)

4. Longest arithmetic subsequence.

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

LONGEST-ARITHMETIC-SUBSEQUENCE(A, d): Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest subsequence S of A such that $S[i + 1] - S[i] = d$ for each i .

Recursive case:

Let $END_LAS(i)$ be the length of the longest arithmetic subsequence (LAS) ending at element $S[i]$.

$S[i]$ is the last element, but what about elements before? Elements before the last element in the subsequence are also arithmetic subsequences but ending at a different element. We want to find the element $S[j]$ among these that produces the longest LAS and satisfies all constraints: that $1 \leq j < i$ and $S[i] - S[j] = d$

$$END_LAS(i) = 1 + \max_{1 \leq j < i: S[i] - S[j] = d} \{END_LAS(j)\}$$

Base case:

$i \leq 1$: In this case, we are ending at the first element, so it is not possible to have a prepending LAS. Thus

$$END_LAS(i) = 1$$

Getting the final answer:

We now have END_LAS , but we're trying to find LAS! To get the final answer, recall that any LAS ends with some element in the array, so it must be accounted for with some index in END_LAS . To find the longest LAS, iterate through the table created by END_LAS and see which ending index produces the maximal value.

Additional Slides

Useful Notations

Sum Σ / product Π

Over an array
 $A[1, \dots, n]$

$$\sum_{i=1}^n A[i] = A[1] + A[2] + \dots + A[n]$$

$$\prod_{i=1}^n A[i] = A[1] \cdot A[2] \cdot \dots \cdot A[n]$$

Over a set $S = \{s_1, \dots, s_n\}$

$$\sum_{s \in S} s \quad \prod_s s$$

Conditional

$$\sum_{i: i \text{ is even}} A[i] \quad \prod_{s \in S: s > 0} s$$

Max/ min

Over an array
 $A[1, \dots, n]$

$$\max_{i: 1 \leq i \leq n} A[i]$$

Over a set $S = \{s_1, \dots, s_n\}$

$$\min_{s \in S} s$$

Sets Operations

Union

$$\bigcup_{i=1}^n S_i = S_1 \cup S_2 \cup \dots \cup S_n$$

Intersection

$$\bigcap_{i=1}^n S_i = S_1 \cap S_2 \cap \dots \cap S_n$$

Set minus

$$A \setminus B = A \cap \bar{B}$$

It is common to write $S \leftarrow S \setminus \{s\}$ for
"remove element s from set S "

Big AND $\wedge$ / big OR $\vee$

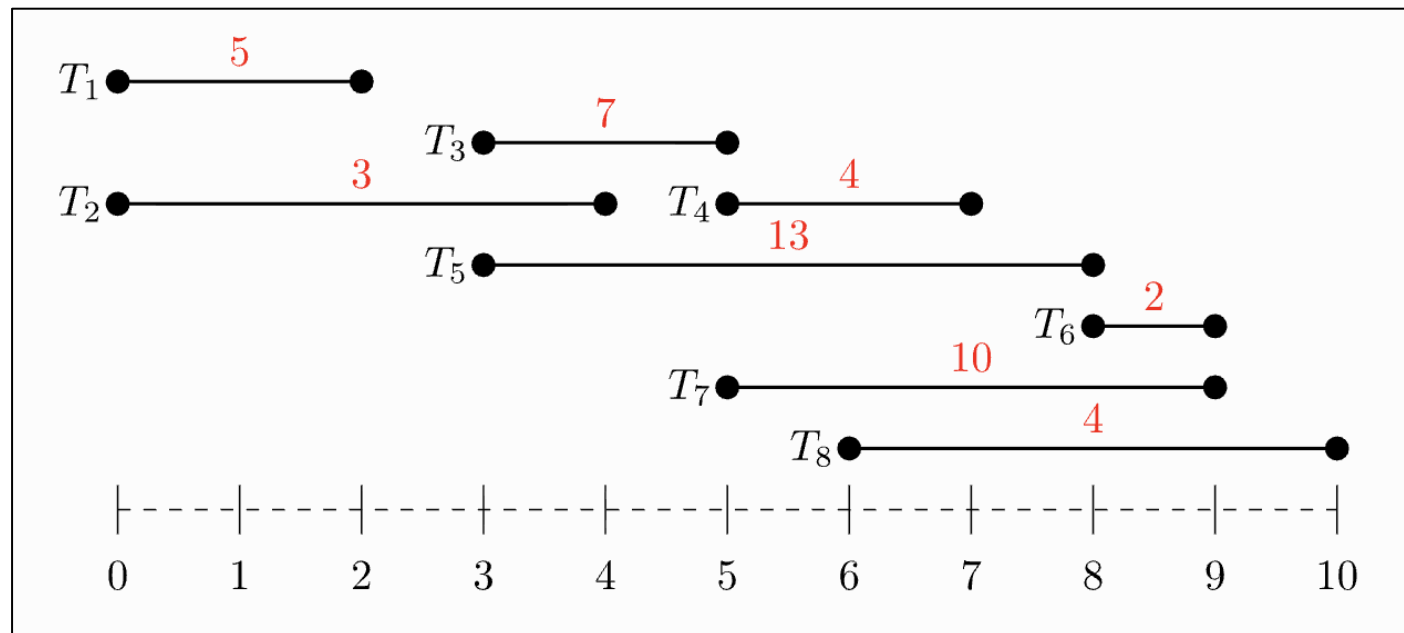
Over an array
of truth values
 $T[1, \dots, n]$

$$\bigwedge_{i=1}^n T[i] = T[1] \wedge T[2] \wedge \dots \wedge T[n]$$

$$\bigvee_{i=1}^n T[i] = T[1] \vee T[2] \vee \dots \vee T[n]$$

Weighted Task Selection

- Problem: given a list of n tasks $T_1, \dots, T_n$ each with a start time s_i , finish time f_i , and value v_i , find the optimal set of **non-overlapping** tasks that **maximizes the total value** of the selected tasks
 - Assume tasks are sorted by their finishing time
 - In the picture below, the optimal set of tasks is $\{T_1, T_3, T_7\}$ with a total value of 22



Weighted Task Selection (continued)

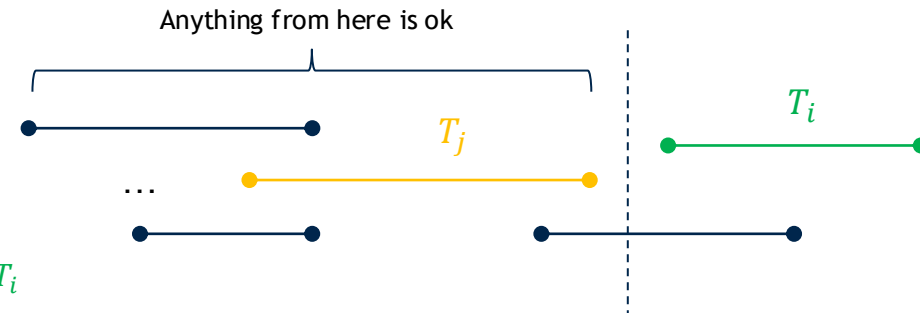
▶ DP Recipe #1:

- ▶ Let $OPT(i)$ be the **optimal value that can be obtained by using only the first i elements of the list** (i.e., $T_1, \dots, T_i$)

▶ DP Recipe #2:

- ▶ Consider just the i th task T_i . There are 2 possibilities:

- ▶ Case 1: T_i is in the optimal set of tasks. Then we should **use it!**
 - ▶ We must make sure the rest of the elements in the set do not conflict with T_i
 - ▶ Idea: take the optimal set that uses only tasks from $T_1, \dots, T_j$, where T_j is the element with the latest finishing time that does not conflict with T_i . **Important conclusion:** no elements from this set will conflict with T_i
 - ▶ $OPT(i) = v_i + OPT(j)$, where j is the max index such that T_i and T_j don't overlap
- ▶ Case 2: T_i is not in the optimal set of tasks. Then we should **lose it!**
 - ▶ Same as just looking at the first $i - 1$ tasks. So $OPT(i) = OPT(i - 1)$
- ▶ Putting everything together: we'll take the max of the 2 possible cases
 - ▶ Base case: no tasks ($n = 0$) has total weight 0



$$OPT(i) = \max\{OPT(i - 1), v_i + OPT(j)\}$$

for the maximum j such that $f_j < s_i$

Weighted Task Selection (continued)

- ▶ DP Recipe #3:
 - ▶ Allocate memo table of n elements and fill it up
- ▶ DP Recipe #4:
 - ▶ We have the optimal value, but how do we get the actual set of selected tasks?
 - ▶ **Backtracking idea**: have a separate “backtrack” table where for every entry, we have a pointer to the previous entry we came from (either $i - 1$ or j)

Index i	0	1	2	3	4	5	6	7	8
$OPT(i)$	0	5	5	12	16	18	20	22	22



- ▶ Using backtracking info: coming from $OPT(j)$ means that we have included that T_i in our set! Whenever we backtrack to $OPT(j)$, then the element we jumped from is in our solution set

WEIGHTEDTASKSELECTION($T[1, \dots, n]$):

```
allocate table[n] and backtrack[n]
table[0] = 0
for i = 1 to n do:
    find min  $j < i$  s.t.  $f_j \leq s_i$ 
    table[i] = max{table[i-1],  $v_i + \text{table}[j]$ }
    if table[i] = table[i - 1]:
        backtrack[i] = i - 1
    else
        backtrack[i] = j
return (table, backtrack)
```

- ▶ DP Recipe #5:
 - ▶ n entries each taking $O(\log n)$ to fill (can use binary search to find j)
 - ▶ So $O(n \log n)$

Longest Increasing Subsequence

- ▶ Problem: Given a sequence of S of numbers, find the **longest increasing subsequence** composed of elements of strictly increasing order
 - ▶ Subsequence elements don't need to be continuous, but must be taken in their original order
 - ▶ In the sample sequence below, we have several longest increasing subsequences (LIS)
 - ▶ (0, 8, 12, 14)
 - ▶ (0, 8, 10, 14)
 - ▶ (0, 7, 12, 14)
 - ▶ (0, 7, 10, 14)
 - ▶ (0, 5, 10, 14)

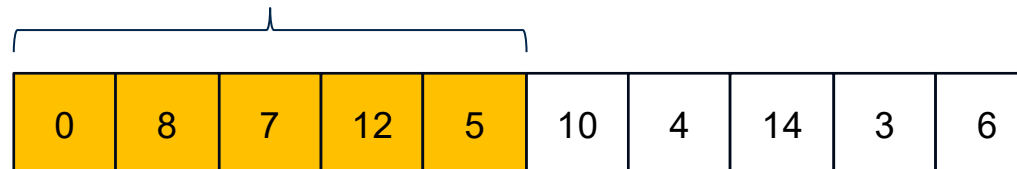
0	8	7	12	5	10	4	14	3	6
---	---	---	----	---	----	---	----	---	---

Longest Increasing Subsequence (continued)

- ▶ DP Recipe #1:

- ▶ When making a longer LIS from a shorter LIS, we need to know what the **last element** of the shorter LIS is so we can maintain the invariant that elements are strictly increasing
 - ▶ Idea: Restrict our view of the problem so that we're considering only subsequences of $S[1, \dots, i]$ that **end with $S[i]$**
- ▶ Let $END_LIS(i)$ be the **length of a longest subsequence ending with $S[i]$**

Discuss: What is the length of a (general) LIS of $S[1, \dots, 5]$? How does it compare with $END_LIS(5)$?



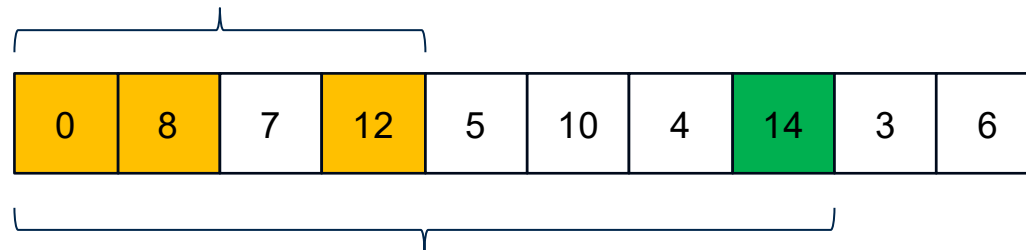
0	8	7	12	5	10	4	14	3	6
---	---	---	----	---	----	---	----	---	---

Longest Increasing Subsequence (continued)

► DP Recipe #2:

- Consider just the i th element $S[i]$. We know that $END_LIS(i)$ must end with $S[i]$
- Furthermore, everything preceding $S[i]$ in $END_LIS(i)$ is also an increasing subsequence
 - Necessary condition: the increasing subsequence ends with an element $S[j]$ that comes before $S[i]$ and it is strictly less than $S[i]$. Otherwise $END_LIS(i)$ would not be a valid increasing subsequence
 - Necessary condition: the increasing subsequence is a **longest** increasing subsequence ending with $S[j]$. Otherwise $END_LIS(i)$ would not be a longest increasing subsequence
- Putting it all together: we'll take j that yields the max $END_LIS(j)$
 - Base case: sequence of 0 numbers ($i = 0$) has END_LIS length 0
 - To get from END_LIS back to original LIS: take the max END_LIS out of all possible ending values

LIS ending with $S[4] = 12$, which comes before and is strictly less than $S[8] = 14$

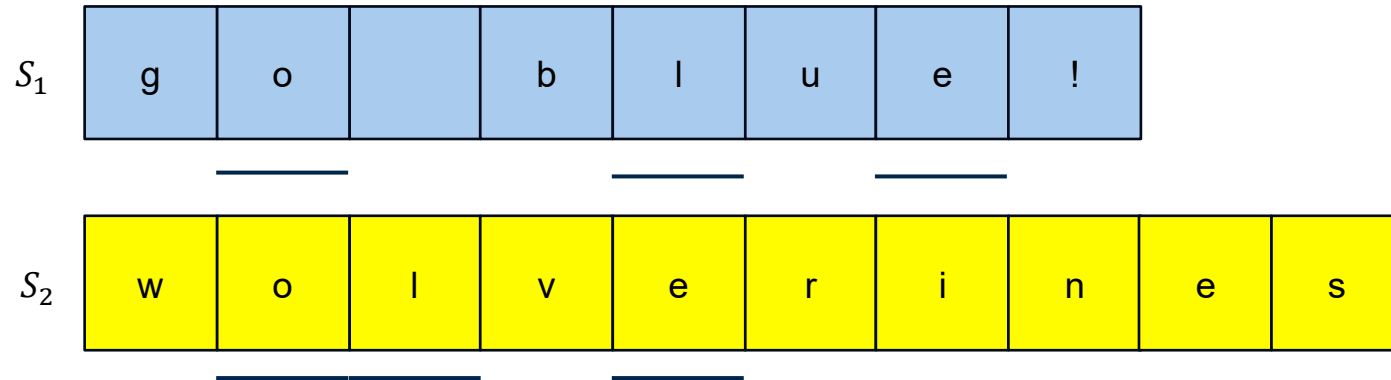


$END_LIS(8)$ ending with $S[8] = 14$

$$END_LIS(i) = 1 + \max\{END_LIS(j)\} \\ \text{such that } 0 \leq j < i \text{ and } S[j] < S[i]$$

Longest Common Subsequence

- ▶ Problem: Given 2 strings S_1 and S_2 , find the **longest common subsequence**
 - ▶ Subsequence elements don't need to be continuous, but must be taken in their original order
 - ▶ Given S_1 ="go blue!" and S_2 ="wolverines", a longest common subsequence (LCS) is "ole"



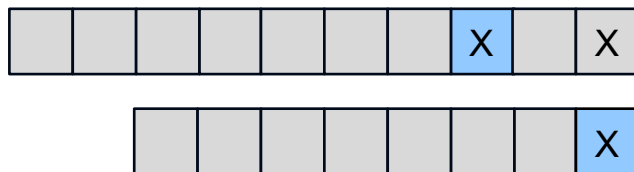
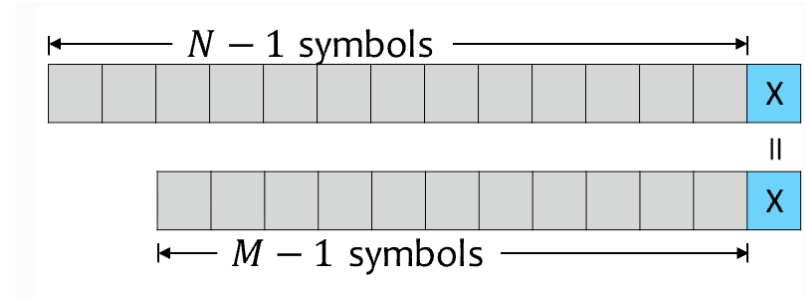
Longest Common Subsequence (continued)

- ▶ DP Recipe #1:
 - ▶ Unlike in LIS, we do not need to know the last element of a smaller LCS to make a bigger LCS
 - ▶ As long as the elements we append are **common to both strings**, we can create a valid bigger LCS
 - ▶ Let $LCS(S_1, S_2)$ be the length of a **longest common subsequence between S_1 and S_2**

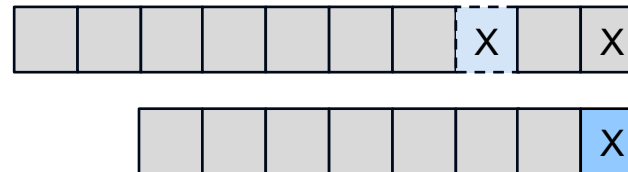
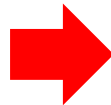
Longest Common Subsequence (continued)

▶ DP Recipe #2:

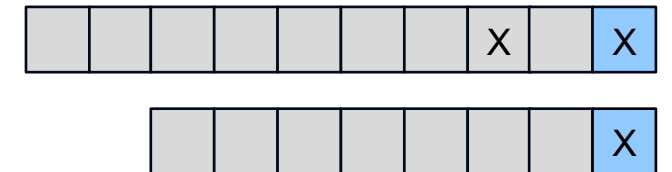
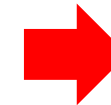
- ▶ Consider just the **last element** of S_1 and S_2 . There are 2 possibilities:
- ▶ Case 1: Last element of S_1 and S_2 are the same. **Use it!**
 - ▶ Take the LCS of $S_1[1, \dots, N - 1]$ and $S_2[1, \dots, M - 1]$ and append the last character $S_1[N] = S_2[M]$ to it
 - ▶ Why is this necessarily a LCS of S_1 and S_2 ?
 - ▶ Suppose for a contradiction that a LCS of S_1 and S_2 does not select either $S_1[N]$ or $S_2[M]$. Then it must not be a LCS since we can create an even longer LCS by appending $S_1[N] = S_2[M]$
 - ▶ Therefore, a LCS of S_1 and S_2 selects **at least one** of $S_1[N]$, $S_2[M]$. If one of $S_1[N]$, $S_2[M]$ is not selected, we can forcibly select it:



One of $S_1[N]$, $S_2[M]$ is not selected



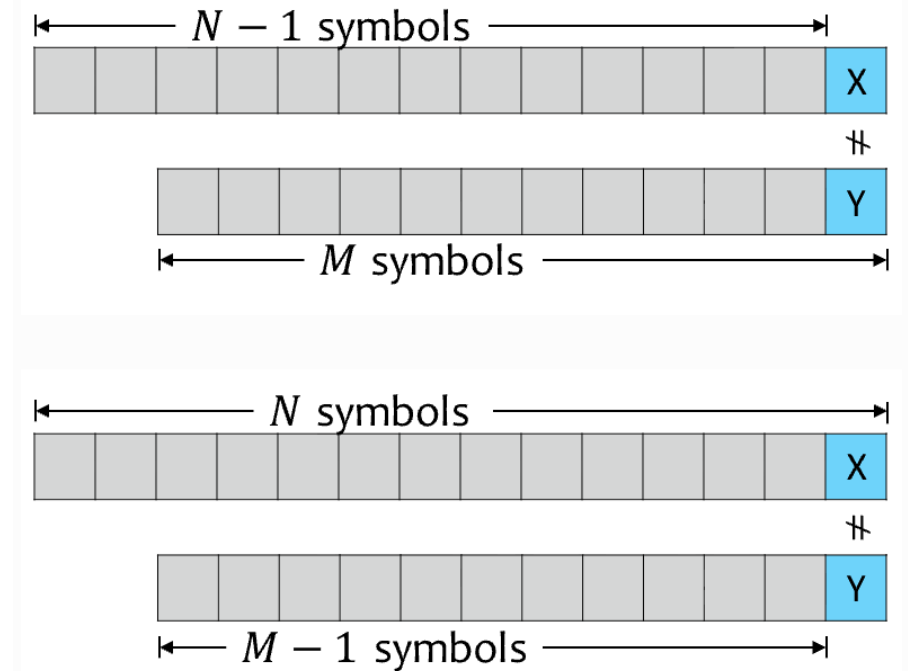
Observe: Element selected is equal to $S_1[N]$, $S_2[M]$.
Unselect it and replace it with $S_1[N]$ or $S_2[M]$



Both $S_1[N]$ and $S_2[M]$ are selected

Longest Common Subsequence (continued)

- ▶ DP Recipe #2:
 - ▶ Case 2: Last element of S_1 and S_2 are not the same.
Lose it!
 - ▶ Same as just looking at $S_1[1, \dots, N - 1]$ or $S_2[1, \dots, M - 1]$ since $S_1[N]$ and $S_2[M]$ cannot both be selected ($S_1[N] \neq S_2[M]$)
 - ▶ Take the max of either $S_1[1, \dots, N - 1]$ against S_2 , or S_1 against $S_2[1, \dots, M - 1]$
 - ▶ Putting it all together: separate into case 1 and case 2
 - ▶ Base cases: LIS between any string of 0 length ($N = 0$ or $M = 0$) is 0



$$LCS(S_1, S_2) = \begin{cases} 1 + LCS(S_1[1, \dots, N - 1], S_2[1, \dots, M - 1]), & S_1[N] = S_2[M] \\ \max\{LCS(S_1[1, \dots, N - 1], S_2), LCS(S_1, S_2[1, \dots, M - 1])\}, & S_1[N] \neq S_2[M] \end{cases}$$